STRING METHODS: upper(), lower(), replace()

Page 1: Introduction to Strings and String Methods

What is a String?

A **string** is a sequence of characters enclosed in quotes. These characters can include letters, numbers, spaces, and symbols.

Example:

```
text = "Hello World"
```

Strings are one of the most commonly used data types in programming because they represent textual data.

What are String Methods?

String methods are **built-in functions** that allow us to perform operations on strings, such as:

- Changing letter case
- Searching text
- Replacing words
- Formatting output

Important Property of Strings

Strings are **immutable**, which means:

- You **cannot change** the original string directly
- All methods return a **new modified string**

Example:

```
text = "hello"
text.upper()
print(text)
```

Output:

```
hello
```

☞ The original string remains unchanged.

Page 2: The `upper()` Method

Definition

The `upper()` method converts all characters in a string to **uppercase letters**.

Syntax

```
string.upper()
```

Example 1

```
text = "hello world"
print(text.upper())
```

Output:

```
HELLO WORLD
```

Example 2

```
text = "Python123"
print(text.upper())
```

Output:

```
PYTHON123
```

☞ Numbers and symbols remain unchanged.

Use Cases of `upper()`

1. **Standardizing input**
2. `name = input("Enter name: ").upper()`
3. **Case-insensitive comparison**
4. `if user_input.upper() == "YES":`
5. `    print("Confirmed")`

Key Points

- Converts all letters to uppercase
- Does not affect numbers or symbols
- Returns a new string

Page 3: The `lower()` Method

Definition

The `lower()` method converts all characters in a string to **lowercase letters**.

Syntax

```
string.lower()
```

Example 1

```
text = "HELLO WORLD"
print(text.lower())
```

Output:

```
hello world
```

Example 2

```
text = "PyThOn"
print(text.lower())
```

Output:

```
python
```

Use Cases of `lower()`

1. User Input Handling

```
email = input("Enter email: ").lower()
```

2. Case-Insensitive Comparison

```
if password.lower() == "admin":
    print("Access granted")
```

Difference Between `upper()` and `lower()`

Method	Output Type
`upper()`	ALL CAPS
`lower()`	all small letters

Key Points

- Converts all letters to lowercase
- Helps in uniform text processing
- Returns a new string

Page 4: The `replace()` Method

Definition

The `replace()` method is used to **replace a specific part of a string with another value**.

Syntax

```
string.replace(old, new)
```

- `old` → text to be replaced
- `new` → replacement text

Example 1

```
text = "I like apples"
print(text.replace("apples", "oranges"))
```

Output:

```
I like oranges
```

Example 2

```
text = "banana"
print(text.replace("a", "o"))
```

Output:

```
bonono
```

☞ All occurrences are replaced.

Example 3 (Limited Replacement)

```
text = "apple apple apple"
print(text.replace("apple", "orange", 2))
```

Output:

```
orange orange apple
```

☞ Only first 2 occurrences are replaced.

Use Cases of `replace()`

1. **Correcting text**
2. **Removing unwanted characters**
3. `text = "hello!!!"`
4. `print(text.replace("!", ""))`
5. **Formatting strings**

Key Points

- Replaces specific text
- Can limit number of replacements
- Returns a new string

Page 5: Combined Usage and Summary

Combining Methods

You can use multiple string methods together:

Example:

```
text = "Hello World"
result = text.lower().replace("world", "python")
print(result)
```

Output:

```
hello python
```

Real-Life Example

```
message = "WELCOME USER!!!"
clean_message = message.lower().replace("!!!", "")
print(clean_message)
```

Output:

```
welcome user
```

Comparison Table

Method	Purpose	Example Output
`upper()`	Convert to uppercase	`"hi"` → `"HI"`
`lower()`	Convert to lowercase	`"HI"` → `"hi"`
`replace()`	Replace text	`"a b"` → `"a c"`

Final Key Takeaways

- Strings are **immutable**
- All methods return **new strings**
- `upper()` → uppercase conversion
- `lower()` → lowercase conversion
- `replace()` → text replacement

Conclusion

These three string methods are **fundamental tools** in programming. They are widely used in:

- Data cleaning
- User input handling
- Text processing
- Formatting outputs

Mastering them helps you write cleaner and more efficient programs.